IndicSent-M

Multimodal Sentiment Analysis for Indian Languages Using Text and Images

Project report · proposed solution · technical stack · feasibility · benefits · research

1. Proposed Solution

1.1 Problem

Indian social-media content is multilingual, code-mixed and visual. A meme's sentiment often lives in the *interaction* between its caption and its picture (a cheerful line over a grim image can be sarcasm), yet most sentiment tools read only text and are built for English. We need a system that reads an Indian-language meme's caption **and** image, classifies it as **positive, neutral or negative**, and shows *why*.

1.2 Solution overview

IndicSent-M is an end-to-end multimodal pipeline. The caption is encoded by a multilingual Transformer (XLM-RoBERTa base); the image by a pretrained CNN (ResNet-50). The two representations are projected to a shared 256-d space and **fused** — either by concatenation (baseline) or by bidirectional **cross-attention**, in which every caption token attends over the 49 regions of the image feature grid and vice versa — before a small classifier outputs the three class probabilities. Text-only and image-only models are trained with the identical recipe so the value of fusion can be measured, not assumed.

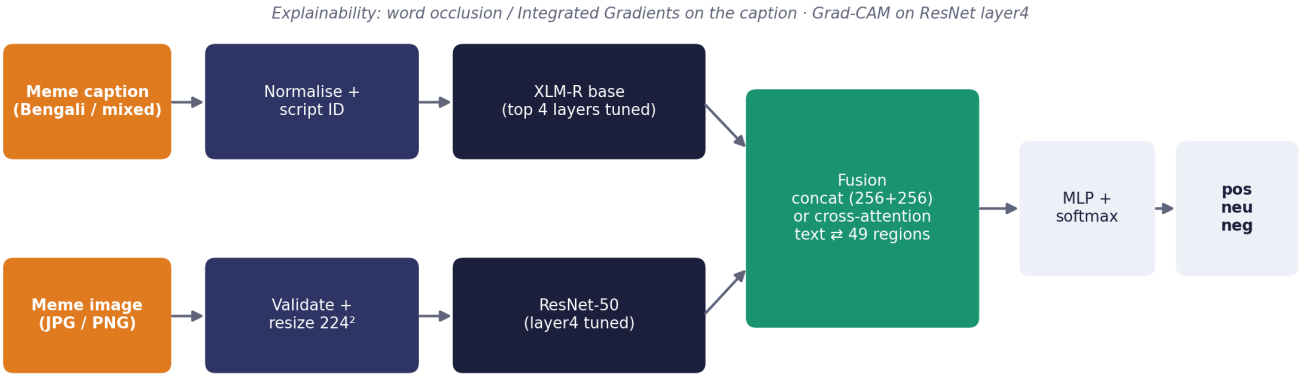


Figure 1. IndicSent-M architecture. Both inputs are mandatory for the multimodal models: the code refuses a missing modality, and unit tests verify that changing only the image or only the text changes the output.

1.3 Solution components

- **Data pipeline.** Loads the MemoSen Bengali meme dataset (4,372 annotations), validates every image, normalises text conservatively (keeps emojis, negations, casing and the ZWJ/ZWNJ joiners Indian scripts need), identifies scripts (Bengali / Latin / code-mixed), removes duplicates and builds a **leakage-safe** split: 3,054 train / 434 validation / 874 test memes.
- **Three model families.** Text-only (XLM-R), image-only (ResNet-50) and multimodal (two fusion variants), partially fine-tuned so they train on an 8 GB laptop.
- **Explainability.** Word-level attributions (leave-one-word-out occlusion; Integrated Gradients with an automatic completeness check) and Grad-CAM heatmaps on the image.
- **Application.** A Streamlit web app (analysis, model comparison, dataset and methodology pages) and a FastAPI REST service; Docker file for deployment; automated tests.

1.4 How a prediction is made

- **Input checks** — the caption must be non-empty (text/multimodal modes); the image must decode, be ≤ 10 MB, ≤ 40 MP and ≥ 32 px per side. Unsupported scripts, emoji-only text and truncation produce warnings.
- **Encoding** — caption → XLM-R tokens (max 96; the 95th-percentile caption is 55 tokens) → mean-pooled 768-d vector and token sequence; image → 224×224 → ResNet-50 → 2048-d vector and 7×7 region grid.
- **Fusion and classification** — concat or cross-attention → MLP → softmax over positive / neutral / negative.
- **Output** — label, confidence (explicitly labelled as the predicted-class probability, not a guarantee), all three probabilities, highlighted words and a Grad-CAM overlay.

1.5 Key design decisions

Decision	Choice	Reason
Text encoder	XLM-R base (not IndicBERT)	Pre-training data covers Bengali and romanised Bengali/Hindi; 21% of captions mix Bengali and Latin script
Image encoder	ResNet-50, ImageNet weights	Strong, small, Grad-CAM-compatible; runs on CPU
Fine-tuning	Top 4 XLM-R layers + ResNet layer4	Fits 8 GB unified memory; limits over-fitting on ~3 k memes; frozen BatchNorm kept in eval mode
Fusion	Concatenation and cross-attention	Simple baseline vs. token-to-region interaction
Class imbalance	√-inverse-frequency loss weights	Labels are 62.5% negative, 30.8% positive, 6.6% neutral
Model selection	Best validation macro-F1, early stopping	Test set never used for choices
Text cleaning	Conservative normalisation	Emojis, negation, casing and joiners carry sentiment or script meaning
Checkpoints	Trained weights only (frozen rebuilt)	~60–200 MB instead of 1.1 GB+; loader refuses partially random models

1.6 Evaluation design

- **Fair comparison** — all models use the same split, preprocessing, optimiser, schedule, epochs budget and selection rule; only the architecture changes.
- **Primary metric: macro-F1** — with this imbalance, always predicting “negative” already gives 62.6% accuracy but only 0.257 macro-F1.
- **Uncertainty** — 95 % bootstrap confidence intervals and paired bootstrap tests between models.
- **Modality ablation** — re-test the fused models with a neutral image or without the caption to prove both inputs are used.
- **Robustness** — a second split that holds out whole *source pages*, because we found the page a meme comes from strongly predicts its label (§4.2).

Leakage control. Near-duplicate memes (same image hash or the same long caption) are grouped with union-find and each group is kept inside a single split; 0 groups span splits. A unit test checks that two runs produce the identical split.

1.7 Implementation status

Component	Status
Data pipeline, leakage-safe and source-disjoint splits	Implemented and tested
Text-only, image-only, concat and cross-attention models	Implemented, trained, evaluated
Statistics (bootstrap CIs, paired tests), modality ablation	Implemented
Occlusion, Integrated Gradients (verified), Grad-CAM	Implemented
Streamlit app, FastAPI service, automated tests	Implemented and tested
Docker image	Dockerfile written; not built in this environment
OCR, Hinglish data, calibration, multiple seeds	Future work

2. Technical Stack

Every component is open-source and runs on a laptop. Versions are those the reported results were produced with (pinned in **requirements.txt**).

Layer	Technology (version)	Role in IndicSent-M
Language	Python 3.13 (3.10+ supported)	All code
Deep learning	PyTorch 2.7.1, torchvision 0.22.1	Models, training loop, MPS/CUDA/CPU
NLP models	Hugging Face Transformers 4.57.6, SentencePiece	XLM-RoBERTa base encoder and tokenizer (FacebookAI/xlm-roberta-base)
Computer vision	torchvision ResNet-50 (IMAGENET1K_V2)	Image encoder, Grad-CAM target layer
Fusion	PyTorch nn.MultiheadAttention, MLP	Concatenation and cross-attention fusion
Data processing	pandas, NumPy, Pillow, openpyxl	Loading, validation, hashing, caching, splits
Evaluation	scikit-learn, SciPy	Metrics, confusion matrices, stratified and source-disjoint splits
Explainable AI	Own implementations	Occlusion, Integrated Gradients (completeness check), Grad-CAM
Visualisation	Matplotlib, Seaborn	Dataset figures, result plots, explanation gallery
Frontend	Streamlit 1.65	Five-page web app
Backend	FastAPI + Uvicorn	REST endpoint /predict, /health
Experiment tracking	CSV / JSON logs	Per-epoch history, metadata with config and environment
Testing	pytest (44 tests), Streamlit AppTest	Data, models, inference, explainers, API, app
Deployment	Docker (python:3.11-slim, CPU wheels)	Containerised app/API; checkpoints mounted
Documents	python-pptx, ReportLab	Slides and this report generated from results

2.1 Hardware and runtime

All results were produced on an **Apple M1 with 8 GB unified memory** using PyTorch's MPS backend; the same code runs on NVIDIA GPUs (CUDA) and on CPU without changes (automatic device selection). Pretrained weights are downloaded once (≈1.1 GB XLM-R, ≈100 MB ResNet-50) and the system then works fully offline.

2.2 Code organisation

- **src/data** — download, preparation, text processing, image validation, PyTorch dataset.
- **src/models** — text, image and multimodal models; factory and checkpoint loader.
- **src/training**, **src/evaluation**, **src/explainability** — shared trainer, metrics and statistics, explainers.
- **app.py**, **src/api.py**, **scripts/** — UI, REST API, one-command pipeline, environment check, document builders.
- **configs/** — one YAML file holds every hyper-parameter; experiment variants inherit from it.

2.3 Why this stack

- **Standard and teachable** — PyTorch and Hugging Face are the de-facto research stack; every model can be swapped by changing one config line.
- **Multilingual by default** — XLM-R's 250 k SentencePiece vocabulary covers Indic scripts and romanised text without custom tokenisation.
- **Lightweight serving** — Streamlit gives an interactive demo in pure Python; FastAPI exposes the same predictor to other systems; Docker packages both.
- **No heavy extras** — explainability methods are implemented directly (≈200 lines) instead of adding large XAI libraries, keeping dependencies small and versions compatible.

3. Feasibility and Viability

3.1 Technical feasibility — demonstrated, not estimated

All four models were trained and evaluated end-to-end on a consumer laptop. Measured costs:

Model	Trainable / total params	Training time	Epochs (best)	Checkpoint	Latency / meme*
Text-only	28.4 M / 277 M	9.6 min	8 (7)	129 MB	19 ms
Image-only	15.0 M / 24 M	10.8 min	8 (7)	57 MB	20 ms
Multimodal (concat)	44.2 M / 302 M	13.7 min	5 (2)	190 MB	39 ms
Multimodal (cross-attn)	44.8 M / 302 M	13.7 min	5 (2)	192 MB	48 ms

*Median at batch size 1 on the M1 GPU (MPS), including image loading, preprocessing and tokenisation.

3.2 Data feasibility

Annotations	Usable memes	Train / val / test	Positive / neutral / negative	Code-mixed captions
4,372	4,362	3,054 / 434 / 874	30.8% / 6.6% / 62.5%	20.7%

MemoSen (LREC 2022) is publicly downloadable and provides images, transcribed captions and 3-class labels; 4,362 of 4,372 annotations are usable after validation. It carries no explicit licence, so it is used for non-commercial research with citation and is not redistributed. Hinglish data (Memotion 3) can be added through a generic CSV adapter once its registration is completed.

3.3 Economic viability

- **Zero licence cost** — every library and both pretrained models are open-source.
- **No special hardware** — training all four models took about 48 minutes of compute on an 8 GB laptop; a cloud GPU would cost well under one hour of rental.
- **Cheap to serve** — CPU-compatible inference, small checkpoints, a single Docker container.

3.4 Operational viability

- Starts even without trained models (shows setup steps; never serves random-weight predictions).
- Handles corrupt, oversized or missing images, empty captions and unsupported scripts with clear messages.
- Reproducible: fixed seed, pinned versions, one-command pipeline, config and environment saved with every model.

3.5 Risks and mitigations

Risk	Mitigation in this project
Small, imbalanced dataset (neutral ≈ 7 %)	Class-weighted loss, macro-F1 as primary metric, early stopping
Shortcut learning (source page predicts label)	Detected; source-disjoint evaluation reported alongside the random split
Over-confident predictions	Confidence labelled as a probability, not a guarantee; calibration listed as future work
Misleading explanations	IG completeness checked and hidden when it fails; limitations shown in the app
Out-of-domain languages	Script detection warns; documentation states the training language

Verdict. The solution is **technically feasible** (built, trained and tested on an 8 GB laptop), **economically viable** (open-source, no paid services, CPU-servable) and **operationally robust** for research and demonstration use. Production use would first need more diverse training data, probability calibration and human review of predictions.

4. Improvements and Benefits

4.1 Measured improvement from multimodal fusion

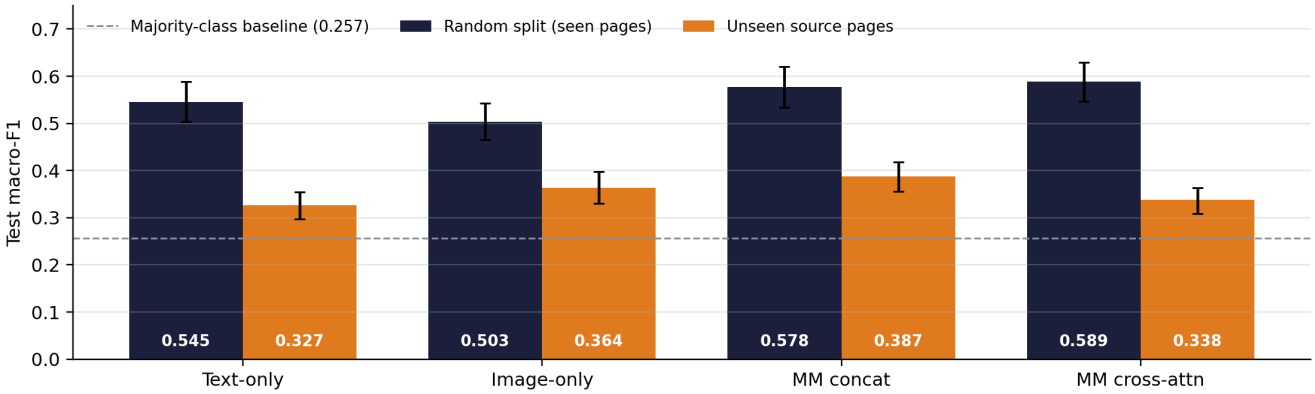


Figure 2. Test macro-F1 with 95 % bootstrap intervals. Random split: 874 test memes; unseen source pages: 938 test memes (different test sets).

Model (random split)	Accuracy	Macro-F1 [95 % CI]	F1 neutral	Δ macro-F1 vs text
Text-only	0.690	0.545 [0.50, 0.59]	0.226	–
Image-only	0.681	0.503 [0.47, 0.54]	0.167	-0.041
Multimodal (concat)	0.743	0.578 [0.53, 0.62]	0.242	+0.033
Multimodal (cross-attn)	0.743	0.589 [0.55, 0.63]	0.256	+0.044

- **Fusion beats both single modalities.** Cross-attention fusion reaches 0.589 macro-F1 vs 0.545 (text-only) and 0.503 (image-only); accuracy rises from 69.0% to 74.3%. The gain over text is +0.044 (95 % CI +0.004 to +0.086) — real but modest; over image-only it is large and clearly significant.
- **Both inputs matter.** Neutralising the image or removing the caption flips 21% / 27% of the cross-attention model's predictions.

4.2 Robustness check and the improvement it motivated

- **Honest generalisation estimate.** The source page alone predicts the label (78% accuracy without reading the meme). On unseen pages, scores drop to 0.327–0.387; concatenation fusion stays best and significantly ahead of text-only (+0.061, CI +0.022 to +0.099).

4.3 Benefits

- **Researchers** — a reproducible, tested baseline for Indian-language multimodal sentiment, with statistical tests and a documented dataset shortcut that future work must control for.
- **Platforms and analysts** — sentiment that accounts for the image, not just the text, in code-mixed posts; explanations make decisions reviewable by humans.
- **Students and educators** — an end-to-end reference implementation (data → models → XAI → app → Docker).
- **Low-resource languages** — shows a laptop-scale route for languages with little labelled data.

4.4 Planned improvements

Hinglish and other Indian-language memes (Memotion 3) · OCR so users need not type captions · CLIP-style vision-language encoders · source-balanced or page-adversarial training against the shortcut · more neutral examples / focal loss · several training seeds and probability calibration · user study of the explanations.

5. Research and References

5.1 Research questions and findings

RQ	Finding (from the experiments)
Does text + image beat a single modality?	Yes. Best random-split model: cross-attention fusion (0.589); both fusion models significantly beat image-only; vs text-only the gain is +0.044 (attention) and +0.033 (concat, not significant).
Concatenation or cross-attention?	Indistinguishable on the random split; on unseen pages concatenation is significantly better (+0.049).
Do scores reflect sentiment understanding?	Partly. A content-free source-page prior reaches 78.3% accuracy; every model loses 0.14–0.25 macro-F1 on unseen pages (92 % of positive training memes come from one page).
Can predictions be explained reliably?	Occlusion is exact; Integrated Gradients failed its completeness check for all text-only gallery examples (5/12 passed overall), so it is shown only when verified.

5.2 Research contribution

(i) A leakage-safe, fully reproducible comparison of text, image and two fusion models for Bengali meme sentiment with confidence intervals and paired significance tests; (ii) identification and quantification of a source-page shortcut in MemoSen and a source-disjoint protocol to measure it; (iii) an explainability layer that verifies Integrated Gradients' completeness before showing it; (iv) an open, tested application. Full details: **docs/research_results.md**.

Method followed: literature and dataset survey → leakage-safe data preparation → identical training of four models → evaluation with uncertainty → ablation → robustness split → explanation analysis. Every number in this report is read from the saved evaluation files.

5.3 References

[1] E. Hossain, O. Sharif, M. M. Hoque. MemoSen: A Multimodal Dataset for Sentiment Analysis of Memes. *Proc. LREC 2022*, pp. 1542–1554. aclanthology.org/2022.lrec-1.165

[2] A. Conneau, K. Khandelwal, N. Goyal, et al. Unsupervised Cross-lingual Representation Learning at Scale. *ACL 2020*, pp. 8440–8451. aclanthology.org/2020.acl-main.747

[3] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. *CVPR 2016*, pp. 770–778.

[4] A. Vaswani, N. Shazeer, N. Parmar, et al. Attention Is All You Need. *NeurIPS 2017*.

[5] D. Kakwani, A. Kunchukuttan, S. Golla, et al. IndicNLPSuite: Monolingual Corpora, Evaluation Benchmarks and Pre-trained Multilingual Language Models for Indian Languages. *Findings of EMNLP 2020*, pp. 4948–4961.

[6] S. Mishra, S. Suryavardan, P. Patwa, et al. Memotion 3: Dataset on Sentiment and Emotion Analysis of Codemixed Hindi-English Memes. arXiv:2303.09892, 2023.

[7] C. Sharma, D. Bhageria, W. Scott, et al. SemEval-2020 Task 8: Memotion Analysis – the Visuo-Lingual Metaphor! *Proc. SemEval-2020*, pp. 759–773.

[8] M. Sundararajan, A. Taly, Q. Yan. Axiomatic Attribution for Deep Networks. *ICML 2017*.

[9] R. R. Selvaraju, M. Cogswell, A. Das, et al. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. *ICCV 2017*.

[10] J. Li, W. Monroe, D. Jurafsky. Understanding Neural Networks through Representation Erasure. arXiv:1612.08220, 2016.

[11] T. Berg-Kirkpatrick, D. Burkett, D. Klein. An Empirical Investigation of Statistical Significance in NLP. *EMNLP-CoNLL 2012*, pp. 995–1005.

[12] I. Loshchilov, F. Hutter. Decoupled Weight Decay Regularization. *ICLR 2019*.

[13] D. Kiela, H. Firooz, A. Mohan, et al. The Hateful Memes Challenge: Detecting Hate Speech in Multimodal Memes. *NeurIPS 2020*.

Entries [1], [2], [5]–[7], [10], [11] were checked against the ACL Anthology / arXiv; the others are standard references cited from the published record (see docs/references.md).